SGEF

Sistema de Gestão e Educação Financeira

Documentação Técnica — Script ETL

Engenharia de Dados — Projeto Integrador

Membros do Projeto

Lucas Olivera De Carvalho Mendes · Artur Machado Máximo · Arthur Prados Piana

Felipe Josué Rodrigues · Ryan Alexandre Cavalcante

Versão 1.0 — 10/05/2026

1. Visão Geral do Processo

O script sgef_etl.py implementa o pipeline completo de Engenharia de Dados do projeto SGEF (Sistema de Gestão e Educação Financeira). O processo segue a arquitetura ETL — Extração, Transformação e Carga — dividida em quatro módulos sequenciais e independentes, garantindo modularidade, rastreabilidade e facilidade de manutenção.

Fontes de dados utilizadas:

- Pesquisa Acadêmica (Google Forms): 70 respostas, 34 colunas — comportamento e educação financeira da população.
- Base de Crédito BCB (Banco Central do Brasil): 1.032.181 registros, 23 colunas — carteira de crédito nacional por UF e modalidade.

Visão geral dos módulos e suas saídas:

Módulo / Script	Função Principal	Saída
01 — Extração & Ingestão extrair_dados_form() extrair_dados_bcb()	Carrega os dois CSVs com detecção automática de encoding e separador. Trata erros de arquivo.	df_bruto_forms df_bruto_bcb
02 — Inspeção & Limpeza tratar_limpar_forms() tratar_limpar_bcb()	Renomeia colunas, remove duplicatas, trata nulos, normaliza strings e converte tipos.	df_forms_limpo df_bcb_limpo
03 — Transformação transformar_dados()	Desmembra os DataFrames em tabelas temáticas e cria features analíticas (groupby, agregações).	7 tabelas modulares (dict)
04 — Validação & Carga estruturar_e_exportar()	Valida colunas críticas via assert e exporta todos os CSVs para a pasta database/.	9 arquivos CSV finais

Módulo 01 Extração & Ingestão

Funções: extrair_dados_form() | extrair_dados_bcb()

2.1 Objetivo

Localizar e carregar os arquivos CSV de entrada na memória como DataFrames pandas brutos, sem nenhuma transformação. Em caso de falha, retornar um DataFrame vazio e exibir mensagem de erro descritiva — garantindo que o processo não termine abruptamente sem diagnóstico.

2.2 Comportamento técnico

- Tenta três encodings em sequência: utf-8 → utf-8-sig → latin-1, parando no primeiro que funcionar.
- O arquivo da Pesquisa usa sep=',' (padrão Google Forms).
- O arquivo do BCB usa sep=';' e decimal=',', padrão de exportações do Banco Central.
- O parâmetro low_memory=False é aplicado ao BCB para evitar inferências de tipo incorretas em arquivos de mais de 1 milhão de linhas.
- Bloco try/except captura FileNotFoundError separadamente de erros de leitura, fornecendo mensagens distintas ao usuário.

2.3 Parâmetros e retorno

Parâmetro / Retorno	Descrição
nome_arquivo (str)	Caminho completo para o arquivo CSV de entrada.
Retorno — sucesso	pd.DataFrame com todos os dados brutos do arquivo.
Retorno — falha	pd.DataFrame() vazio. O módulo de execução principal verifica .empty e interrompe o pipeline com sys.exit(1).

2.4 Saída esperada no console

```
☑ [Forms] Lido com sep=',' enc='utf-8'
  → 70 respostas encontradas no arquivo.
  → 34 colunas detectadas.

☑ [BCB] Lido com sep=';' enc='utf-8'
  → 1,032,181 registros encontrados.
  → 23 colunas detectadas.
```

Módulo 02 Inspeção & Limpeza

Funções: tratar_limpar_forms() | tratar_limpar_bcb()

3.1 Objetivo

Transformar os DataFrames brutos em DataFrames limpos e padronizados, aplicando as estratégias de tratamento necessárias para garantir a qualidade dos dados antes de qualquer análise.

3.2 Limpeza da Pesquisa (tratar_limpar_forms)

Etapa 1 — Normalização automática dos nomes de colunas:

As colunas do Google Forms chegam com frases longas, acentos, espaços e caracteres especiais. O pipeline abaixo normaliza todos os nomes para snake_case ASCII:

```
df.columns
  .astype(str)
  .str.replace('\ufeff', '', regex=False)   # remove BOM
  .str.strip().str.lower()                  # minúsculo
  .str.normalize('NFKD')                    # decompõe acentos
  .str.encode('ascii', errors='ignore')     # remove acentos
  .str.decode('ascii')
  .str.replace(r'\W+', '_', regex=True)    # espaços → _
  .str.strip('_')                           # remove _ inicial/final
```

Etapa 2 — Renomeação explícita (31 colunas):

Após a normalização automática, as colunas recebem nomes curtos e semânticos via dicionário de mapeamento. Exemplos:

Nome original (normalizado)	Nome final
carimbo_de_data_hora	timestamp
qual_sua_faixa_etaria	faixa_etaria
qual_sua_renda_mensal_aproximada	renda_mensal
voce_possui_uma_reserva_de_emergencia	tem_reserva_emergencia
nota_educ_financeira (numérico escala 1–5)	nota_educ_financeira
atualmente_voce_se_encontra_em_situacao_de...	esta_inadimplente
com_que_frequencia_voce_enfrenta_dificuldades...	freq_dificuldade_pagamento

Etapa 3 — Remoção de duplicatas:

- df.drop_duplicates() — nenhuma duplicata encontrada nos dados atuais (0 removidas).

Etapa 4 — Tratamento de nulos:

Colunas condicionais (ex: 'Se sim, qual?') só são preenchidas pelos respondentes que marcaram 'Sim' na pergunta anterior. O preenchimento com 'Não se aplica' é a estratégia correta, pois ausência de resposta é semanticamente diferente de dado faltante.

- Colunas tratadas: tipo_investimento, dificuldade_aprendizado, desejo_sistema e demais com 'se_sim' ou 'experiencia' no nome.

Etapa 5 — Normalização de strings categóricas:

- .str.strip().str.title() em colunas de perfil (faixa_etaria, renda_mensal, esta_inadimplente...).
- .str.strip() em colunas de múltipla escolha (fontes_informacao, temas_dificeis...).

Etapa 6 — Conversão de tipos:

- timestamp → pd.to_datetime(..., dayfirst=True) para datas no formato DD/MM/AAAA do Google Forms.

3.3 Limpeza do BCB (tratar_limpar_bcb)

Problema encontrado	Estratégia aplicada
Colunas monetárias em string com vírgula decimal	Remoção de '.' (milhar) → troca ',' por '.' → pd.to_numeric(..., errors='coerce').fillna(0.0)
numero_de_operacoes com valor '<= 15' (sigilo BCB)	Substituição por 0 antes da conversão numérica.
data_base como string (YYYY-MM-DD)	pd.to_datetime(..., errors='coerce')
10.766 nulos na coluna 'sr'	Mantidos — coluna de segmentação regulatória, nulo indica não aplicável.
Duplicatas	Nenhuma encontrada (0 removidas).

Módulo 03 Transformação & Features

Função: transformar_dados()

4.1 Objetivo

Aplicar o Princípio da Responsabilidade Única: desmembrar os DataFrames limpos em tabelas temáticas independentes e criar features analíticas por meio de agrupamentos (groupby) e agregações. Cada tabela tem um propósito bem definido e pode ser consumida de forma isolada.

4.2 Tabelas geradas

Da Pesquisa Acadêmica (4 tabelas):

Tabela	Colunas principais	Finalidade
tabela_perfil_pessoal	timestamp, faixa_etaria, situacao_profissional, renda_mensal, tem_reserva_emergencia, faz_investimento, tipo_investimento	Perfil demográfico e financeiro dos respondentes.
tabela_comportamento	controla_gastos, como_controla_gastos, freq_registro_gastos, ultrapassa_orcamento, planeja_gastos, maior_dificuldade_dinheiro, freq_dificuldade_pagamento	Hábitos e comportamentos financeiros.
tabela_educacao	nota_educ_financeira, interesse_aprender, fez_curso_financas, fontes_informacao, temas_dificeis, formato_aprendizado, dashboard_ajudaria, interesse_app_sgef (13 colunas)	Nível e interesse em educação financeira — base para os conteúdos do SGEF.
tabela_inadimplencia_forms	faixa_etaria, renda_mensal, esta_inadimplente, tipos_pendencias, freq_dificuldade_pagamento	Situação de inadimplência declarada pelos respondentes.

Do BCB — dados de crédito (2 tabelas agregadas):

Tabela	Agrupamento / Feature	Finalidade
tabela_credito_por_uf	groupby('uf') → total_operacoes, carteira_ativa_total, inadimplida_total, ativo_problematico_total +	Panorama do crédito e inadimplência por estado — 27 UFs.

	taxa_inadimplencia_pct + taxa_ativo_problematico_pct	
tabela_credito_por_modalidade	groupby(['modalidade','porte']) → carteira_ativa_total, inadimplida_total + taxa_inadimplencia_pct	Inadimplência por tipo de produto financeiro e porte do tomador — 108 segmentos.

Feature cruzada — Pesquisa x BCB (1 tabela):

Tabela	Agrupamento / Feature	Finalidade
tabela_perfil_por_renda	groupby('renda_mensal') → total_respondentes, pct_inadimplentes, pct_tem_reserva, pct_faz_investimento, pct_usou_app, media_nota_educ	Cruzamento entre renda declarada e indicadores financeiros — insumo direto para dashboards e relatório de insights.

4.3 Cálculo das features de inadimplência

A taxa de inadimplência é calculada conforme a fórmula:

```
taxa_inadimplencia_pct = (inadimplida_total / carteira_ativa_total) * 100
```

Para evitar divisão por zero, carteira_ativa_total = 0 é substituída por NaN antes da divisão (.replace(0, np.nan)), resultando em NaN (ao invés de erro ou infinito) para UFs sem carteira ativa.

Módulo 04 Validação & Exportação

Função: estruturar_e_exportar()

5.1 Objetivo

Garantir a integridade dos dados antes de persistir e exportar todos os artefatos do pipeline para a pasta database/, prontos para análise, dashboards e relatórios de BI.

5.2 Validações (assert)

Antes de qualquer exportação, o módulo executa validações programáticas que interrompem o processo imediatamente se alguma condição crítica não for atendida:

Validação	O que verifica
assert not df_forms_limpo.empty	Pesquisa não pode estar vazia após limpeza.
assert not df_bcb_limpo.empty	Base BCB não pode estar vazia após limpeza.
assert len(tabelas) > 0	Pelo menos uma tabela deve ter sido gerada.
assert 'renda_mensal' in df_forms_limpo.columns	Coluna crítica para o cruzamento por renda.
assert 'esta_inadimplente' in df_forms_limpo.columns	Coluna crítica para análise de inadimplência.
assert 'nota_educ_financeira' in df_forms_limpo.columns	Coluna crítica para o score de educação financeira.
assert 'carteira_ativa' in df_bcb_limpo.columns	Coluna base para todos os cálculos de taxa.
assert 'uf' in df_bcb_limpo.columns	Coluna de agrupamento geográfico.
assert 'data_base' in df_bcb_limpo.columns	Coluna de referência temporal da base BCB.

5.3 Arquivos exportados

Todos os arquivos são salvos em formato CSV com sep=';' e encoding='utf-8-sig' (compatível com Excel sem erro de acentuação):

Arquivo CSV	Linhas	Descrição
forms_limpo.csv	70	Base da pesquisa após toda a limpeza.
bcb_limpo_AAAA-MM-DD.csv	1.032.181	Base BCB limpa com data do processamento no nome.

tabela_perfil_pessoal.csv	70	Perfil demográfico dos respondentes.
tabela_comportamento.csv	70	Hábitos financeiros dos respondentes.
tabela_educacao.csv	70	Educação financeira dos respondentes.
tabela_inadimplencia_forms.csv	70	Inadimplência declarada pelos respondentes.
tabela_credito_por_uf.csv	27	Crédito e inadimplência por estado (BCB).
tabela_credito_por_modalidade.csv	108	Crédito por modalidade e porte (BCB).
tabela_perfil_por_renda.csv	5	Feature cruzada: renda x indicadores financeiros.

6. Dependências e Configuração

6.1 Bibliotecas Python

Biblioteca	Uso no script
pandas	Leitura de CSV, manipulação de DataFrames, groupby, to_csv.
numpy	Substituição de 0 por NaN nos cálculos de taxa (np.nan).
os	Construção de caminhos de arquivo (os.path.join) e criação de diretórios.
sys	Encerramento controlado do processo em caso de falha (sys.exit(1)).
datetime.date	Inserção da data atual no nome do arquivo BCB exportado.

Instalação em ambiente local:

```
pip install pandas numpy
```

No Google Colab, pandas e numpy já estão pré-instalados.

6.2 Estrutura de arquivos esperada

```
🗀 minha_pasta/
   ├── sgef_etl.py
   ├── respostas_formulario.csv        ← Pesquisa Acadêmica
   └── planilha_202506.csv             ← Base BCB
```

Após a execução, a pasta database/ é criada automaticamente:

```
🗀 minha_pasta/
   └── database/
       ├── forms_limpo.csv
       ├── bcb_limpo_2026-05-10.csv
       ├── tabela_perfil_pessoal.csv
       ├── tabela_comportamento.csv
       ├── tabela_educacao.csv
       ├── tabela_inadimplencia_forms.csv
       ├── tabela_credito_por_uf.csv
       ├── tabela_credito_por_modalidade.csv
       └── tabela_perfil_por_renda.csv
```

6.3 Como executar

Ambiente local (terminal):

```
python sgef_etl.py
```

Google Colab (célula de notebook):

```
import os
os.chdir('/content')
!python sgef_etl.py
```

7. Fluxo de Execução

O script executa os módulos de forma linear e sequencial. A falha em qualquer etapa de extração interrompe o processo imediatamente, evitando que dados corrompidos avancem no pipeline.

Etapa	Ação
1. Configuração	Define caminhos ARQUIVO_FORMS e ARQUIVO_BCB. Cria a pasta database/ se não existir.
2. Extração Forms	extrair_dados_form() → df_bruto_forms. Se vazio: sys.exit(1).
3. Extração BCB	extrair_dados_bcb() → df_bruto_bcb. Se vazio: sys.exit(1).
4. Limpeza Forms	tratar_limpar_forms(df_bruto_forms) → df_forms_limpo.
5. Limpeza BCB	tratar_limpar_bcb(df_bruto_bcb) → df_bcb_limpo.
6. Transformação	transformar_dados(df_forms_limpo, df_bcb_limpo) → dict com 7 tabelas.
7. Validação	9 asserts verificam integridade dos dados finais.
8. Exportação	9 CSVs salvos em database/ com relatório de linhas no console.

SGEF — Sistema de Gestão e Educação Financeira